

Week 7: Using Flutter Packages

Dr M AOUE

Lecture notes

Objectives. By the end of this lecture you will be able to find a suitable package; add runtime and development dependencies; explain version constraints, transitive dependencies and the lockfile; fetch HTTP data; create and test a reusable Dart package; distinguish ordinary exports from library parts; prepare a package for sharing; configure a Google Maps plugin; request a foreground location; and add markers to a map.

This lecture follows *Using Flutter Packages*, with updated examples and corrections. The programs are in `code/week07/examples` and `code/week07/maps_examples`. Each entry point states what it teaches, how to run it and the expected result. Like week 6, the lecture explains the important lines while the full programs live in the repository. The separate run guide gives the commands and platform setup.

1 Packages and plugins

A **package** groups reusable libraries and metadata. A pure Dart package can provide calculations or HTTP utilities without Flutter widgets. A Flutter package can provide widgets. A **plugin** connects Dart code to platform implementations, for example location services or native maps. The same Dart API does not guarantee that every platform is supported.

Example	Role in this week
<code>intl</code>	Number formatting with an explicit locale.
<code>http</code>	Sending requests and receiving responses.
<code>flutter_lints</code>	Development-time analyzer rules.
<code>area</code>	Our local calculation and formatting package.
<code>google_maps_flutter</code>	Platform map view with camera and markers.
<code>location</code>	Device location and permission handling.

Search pub.dev, then inspect the package's README, supported platforms, SDK requirements, publisher, license, release history, examples and issue tracker. Popularity alone does not establish suitability. Check whether a plugin needs native configuration, an API key, billing or runtime permissions.

2 Add and import a dependency

From the application directory, run:

```
1 flutter pub add http
2 flutter pub add intl
3 flutter pub get
```

`pub add` selects a compatible version and updates the pubspec. It may choose an older release when the SDK or other constraints exclude the newest one. `pub get` resolves dependencies and

downloads missing packages; it does not imply upgrading everything to the newest available release.

The pubspec is YAML: use spaces and preserve indentation. A dependency is a sibling of `flutter`, not a child of `sdk`. Our checked application declares:

```
1 dependencies:
2   flutter:
3     sdk: flutter
4   area:
5     path: packages/area
6   intl: ^0.20.3
7   http: ^1.6.0
8 dev_dependencies:
9   flutter_test:
```

Adding a dependency and importing a library are different steps. A Dart file imports the public library it uses:

```
1 import 'package:http/http.dart' as http;
2 import 'package:intl/intl.dart';
```

An import brings names into scope; it does not download the package. The `http` prefix avoids naming conflicts and makes calls such as `http.Client()` easy to identify.

2.1 Version constraints and the lockfile

Versions have major, minor and patch components. A caret gives a compatible range; for pre-1.0 packages that range is narrower.

Constraint	Allowed range
<code>~1.6.0</code>	At least 1.6.0 and below 2.0.0.
<code>~0.13.5</code>	At least 0.13.5 and below 0.14.0.
<code>~0.0.3</code>	At least 0.0.3 and below 0.0.4.

The pubspec expresses constraints; `pubspec.lock` records exact resolved versions, including transitive dependencies. Commit the application's lockfile so classmates resolve the same starting set. Run `flutter pub outdated` to inspect available updates, then review changes and rerun checks after an intentional upgrade. The versions in this repository are compatible with the SDK used for verification, not a promise that they are the latest releases.

3 Example 00: format numbers with intl

Formatting controls presentation, not the stored number. Explicit locales make the expected output repeatable. The same numeric value uses a comma as the thousands separator in US formatting and a decimal comma in German formatting.

```
1 NumberFormat('#,##0.00', 'en_US').format(1234.567);
2 // 1,234.57
3 NumberFormat('#,##0.00', 'de_DE').format(1234.567);
4 // 1.234,57
5 NumberFormat('0.####', 'en_US').format(12.34567);
6 // 12.3457
```

0 requires a digit; # makes a digit optional. Our area display uses at most four decimal places. Run `lib/00_number_format.dart` and compare all three labels.

4 Example 01: use http and decode JSON

An HTTP request is asynchronous. Build a `Uri`, await the response, check its status, decode the body and verify the expected structure. A successful transport response does not guarantee that the JSON has the fields your app expects.

```

1  final response = await client
2    .get(Uri.https('jsonplaceholder.typicode.com', '/posts/1'))
3    .timeout(const Duration(seconds: 10));
4  if (response.statusCode != 200) {
5    throw Exception('HTTP ${response.statusCode}');
6  }
7  final data = jsonDecode(utf8.decode(response.bodyBytes));
8  if (data is! Map<String, dynamic> || data['title'] is! String) {
9    throw const FormatException('Expected a JSON object with a title.');
```

The service uses an injected `http.Client`. Tests can provide a `MockClient` instead of contacting a real server. The page owns a real client when it creates one and closes it in `dispose`; an injected client belongs to the caller.

4.1 Loading, success and failure

The page stores its future in `initState`. It does not create a fresh request on every build. A typed `FutureBuilder<Post>` displays a progress indicator while waiting, a title on success, and an error with a Retry button on failure. Retry replaces the stored future.

```

1  _post = _service.fetchPost(); // once in initState
2
3  // Retry button:
4  onPressed: () {
5    setState(() { _post = _service.fetchPost(); });
6  },
```

The ten-second timeout limits how long the app waits; it is not a general guarantee that the underlying transport has been cancelled. Never assume the public demonstration endpoint is always available.

Platform setup. Android release builds need the Internet permission in the main manifest. macOS networking needs the appropriate client entitlement. Browser requests are subject to CORS. Prefer HTTPS; an HTTP package does not bypass platform or browser restrictions.

5 Development dependencies and lints

Testing and linting tools belong under `dev_dependencies`. Add the recommended Flutter lint set with:

```
1 flutter pub add --dev flutter_lints
```

In `analysis_options.yaml`, include:

```
1 include: package:flutter_lints/flutter.yaml
```

Run `flutter analyze`. A lint recommends a coding practice; it differs from a compiler error and from a runtime exception. For example, making a widget constructor `const` can satisfy a `const-related` lint when all its arguments are constant.

Important distinction: a root package's dev dependencies can be resolved while developing it, but they are not supplied as dependencies to consumers of that package. Do not import development-only libraries into production code. App size is determined by what code and assets are included, not simply by the number of lines in `dependencies`; unused Dart code can be tree-shaken.

6 Create your own package: the area library

We need no Flutter widgets inside the calculation library, so `area` is a **pure Dart package**. To create a similar project from scratch:

```
1 dart create -t package area
2 cd area
3 dart pub add intl
4 dart pub add --dev test
```

For a reusable widget package, `flutter create --template=package package_name` is appropriate instead. Our supplied package is already at `examples/packages/area`; do not regenerate it over the course files.

6.1 Public API and implementation files

```
1 packages/area/
2   pubspec.yaml
3   README.md
4   CHANGELOG.md
5   lib/area.dart
6   lib/src/shapes.dart
7   lib/src/formatting.dart
8   test/area_test.dart
```

Consumers import `package:area/area.dart`. That entry point exports the public functions. `lib/src` is a convention for implementation code; its path is not a language-enforced access boundary.

```
1 /// Public entry point. Consumers import this file, not files under src.
2 library;
3
4 export 'src/shapes.dart';
5 export 'src/formatting.dart';
```

The calculations return `double` values, so callers can reuse them in further arithmetic. Formatting is separate and returns a `String`. Width and height must be finite and positive; an overflowing product is rejected.

```
1 void _checkDimensions(double width, double height) {
2   if (!width.isFinite || !height.isFinite || width <= 0 || height <= 0) {
3     throw ArgumentError('Dimensions must be finite and positive.');
```

```
4   }
5   if (!(width * height).isFinite) {
6     throw ArgumentError('The area is too large.');
```

```
7   }
8 }
9
10 /// Returns the rectangle area, in square units.
11 double rectangleArea(double width, double height) {
12   _checkDimensions(width, height);
13   return width * height;
14 }
15
```

```

16 /// Returns the triangle area using its base and perpendicular height.
17 double triangleArea(double base, double height) {
18   _checkDimensions(base, height);
19   return base * height / 2;
20 }

```

```

1 import 'package:intl/intl.dart';
2
3 /// Formats a finite value to at most four decimal places in the given locale.
4 String formatArea(double value, {String locale = 'en_US'}) {
5   if (!value.isFinite) throw ArgumentError('Area must be finite. ');
6   return NumberFormat('0.####', locale).format(value);
7 }

```

Keeping validation in the package protects all callers, not just the current UI. Keeping tests checks that later edits preserve its contract.

6.2 Consume a local package

The application points to the directory containing the package's pubspec. The path is relative to the application's pubspec:

```

1 area:
2   path: packages/area

```

Then import and use it:

```

1 import 'package:area/area.dart';
2 final value = rectangleArea(4, 3);
3 final label = formatArea(value); // '12'

```

The package's intl dependency becomes a **transitive dependency** of the app. The calculator only imports area; it does not need a separate intl declaration for that use. Our app nevertheless declares intl directly because the independent formatting example imports it directly.

6.3 Example 02: a calculator using the package

The page owns width and height controllers and disposes them. It parses both values, calls the selected function, formats the result and calls setState. Invalid text is not silently converted to zero.

```

1 final value = triangle
2   ? triangleArea(width, height)
3   : rectangleArea(width, height);
4 setState(() => _result = 'Area: ${formatArea(value)} square units');

```

Run the calculator with width 4 and height 3: rectangle gives 12 square units, triangle gives 6. Try an empty field, a negative value and NaN; all should show a useful message.

7 Several files: export versus part

In our package each source file is an ordinary library with its own imports. The public entry point uses export. This is suitable for independently reusable implementations.

The reading also introduces part and part of. They split **one library**, so imports and private names are shared across its parts. A part belongs to a library, not to an entire package. For comparison, this alternative layout could be used in a separate exercise:

```

1 // area.dart
2 part 'src/rectangle.dart';
3

```

```

4 // src/rectangle.dart
5 part of '../area.dart';
6 double rectangleArea(double width, double height) => width * height;

```

Do not mix this alternative into the supplied exported-library implementation. Code generators sometimes use parts; ordinary imports and exports are usually clearer for this small handwritten package.

8 Share and prepare a package for publication

Dependencies can come from a hosted registry, a local path or Git. A Git dependency may specify a subdirectory and a commit or tag:

```

1 area:
2   git:
3     url: https://example.org/your-team/your-repository.git
4     ref: REPLACE_WITH_A_COMMIT_SHA
5     path: packages/area

```

This URL is a placeholder, not a dependency to paste into the supplied project. A branch can move; a commit identifies a specific revision. A published dependency does not update itself in every consuming app: their constraints and lockfiles still control resolution.

For a publishable package, prepare a meaningful name, description, version, README, changelog, tests, ownership information and a suitable LICENSE. Do not copy someone else's copyright holder or invent a license decision. A BSD license has conditions, including retention of notices; it is not permission to omit all attribution.

```

1 dart analyze
2 dart test
3 dart pub publish --dry-run

```

The dry run validates the proposed archive; it does not publish it. The supplied practice package has `publish_to: 'none'` to prevent publishing. A genuine publication exercise would first remove that guard in a reviewed copy and finish its metadata and license. Actual publishing uses `dart pub publish` and makes the package publicly available. We do not run it for this course example.

9 Google Maps: a plugin needs platform setup

The map apps are a separate project so the package basics do not require credentials. The resolved map plugin supports Android, iOS and web. A location plugin supporting more platforms does not make the combined map app support them too.

1. Add `google_maps_flutter`; generate the target platform runner.
2. Configure a Google Cloud project, billing and the Maps API for each target platform.
3. Create platform-restricted API keys and configure the native or web runner.
4. Run the fixed-coordinate map before adding location requests.

Google Maps usage and pricing depend on the enabled services and billing rules. Consult the official setup and pricing pages; do not treat the chapter's free-use threshold as a current entitlement.

9.1 Platform configuration

- **Android:** enable Maps SDK for Android; place `com.google.android.geo.API_KEY` metadata inside the manifest's application element. Restrict its key to the application ID, certificate fingerprint and API. Add Internet and foreground location permissions where appropriate.

- **iOS:** enable Maps SDK for iOS; supply the key through `GMSServices.provideAPIKey` in the launch configuration. Restrict it to the bundle ID and API. Add a meaningful `NSLocationWhenInUseUsageDescription`.
- **Web:** enable Maps JavaScript API; load its script before the Flutter bootstrap. Restrict the key to allowed HTTP referrers and the API. Browser geolocation requires permission and a secure context such as HTTPS or localhost.

Use the requirements of the resolved plugin version, not the old Android `minSdk 20` or obsolete embedded-view preview flag in the chapter. The README gives the configuration snippets; current native templates may use Kotlin Gradle files and newer application-delegate structures.

9.2 Example 00: fixed map

The initial camera is centered on Beirut. `LatLng` carries latitude and longitude; `CameraPosition` adds zoom. The map receives bounded height through `Expanded` inside a column.

```
1 GoogleMap(
2   initialCameraPosition: const CameraPosition(
3     target: LatLng(33.8938, 35.5018), zoom: 13,
4   ),
5   onMapCreated: (controller) {
6     // Store the controller to move the camera later.
7   },
8 )
```

Changing `initialCameraPosition` is not the normal way to move an existing map. Use the map controller's `animateCamera` or `moveCamera`. Our state stores a nullable controller and disposes it when leaving the screen.

10 Example 01: request foreground location

Checking permission does not request it. Before reading location, verify the device's service status, request activation where supported, then check or request permission. Permanent denial needs a settings change, not repeated prompts. Approximate permission may be sufficient for centering a teaching map.

```
1 if (!await gateway.serviceEnabled() && !await gateway.requestService()) {
2   throw StateError('Enable location services in device settings.');
```

```
3 }
4 var permission = await gateway.hasPermission();
5 if (permission == PermissionStatus.denied) {
6   permission = await gateway.requestPermission();
7 }
8 if (permission != PermissionStatus.granted &&
9     permission != PermissionStatus.grantedLimited) {
10   throw StateError('Location permission is unavailable. Check app settings.');
```

```
11 }
12 return gateway.coordinates().timeout(const Duration(seconds: 15));
13 }
```

`LocationGateway` separates the permission workflow from the device plugin so it can be tested. Its production adapter calls `Location`; the tests provide a fake adapter.

The app requests location only when the user taps its location button. It disables repeated requests while busy and updates the camera after success:

```
1 final point = await findUserLocation(DeviceLocationGateway());
2 if (!mounted) return;
3 await _controller?.animateCamera(
4   CameraUpdate.newLatLngZoom(point, 15),
```

```

5 );
6 if (!mounted) return;

```

Handle asynchronous failures and check `mounted` before updating state. A disabled service or denied permission must not be presented as a successful location fix. Beirut is an explicitly labeled starting point, never a substitute claimed to be the user's position. This exercise uses foreground location and does not enable background tracking.

11 Example 02: markers and info windows

A map's `markers` parameter takes a `Set<Marker>`. Each marker needs a stable `MarkerId` and position. The info window can describe it.

```

1 const Marker(
2   markerId: MarkerId('sample-a'),
3   position: LatLng(33.895, 35.503),
4   infoWindow: InfoWindow(
5     title: 'Sample restaurant A', snippet: 'Teaching data',
6   ),
7 )

```

Our toolbar toggles two invented sample pins near Beirut. They are not the results of a live restaurant search. The combined app can also display a user-position marker with its own ID. Build a new marker set or change the state from which it is built; `setState(() { markers = markers; })` does not itself perform a meaningful data update.

11.1 Going further: live nearby places

The chapter uses a legacy Places nearby-search GET endpoint. A new integration should examine **Places API (New)** and its POST request. A server-side example requests only the fields needed by the app:

```

1 POST https://places.googleapis.com/v1/places:searchNearby
2 Content-Type: application/json
3 X-Goog-API-Key: YOUR_SERVER_KEY
4 X-Goog-FieldMask: places.id,places.displayName,places.location
5
6 {
7   "includedTypes": ["restaurant"],
8   "maxResultCount": 10,
9   "locationRestriction": {
10     "circle": {
11       "center": {"latitude": 33.8938, "longitude": 35.5018},
12       "radius": 1000.0
13     }
14   }
15 }

```

This is an extension design, not a service included in the course code. Store the web-service credential on a backend. A key placed in Dart or supplied with `--dart-define` is still extractable from the app; those mechanisms are not secret storage. Client Maps SDK keys have their own platform restrictions.

The backend should validate the request, use a restricted credential and return a small list of IDs, display names and coordinates. The app checks the response and converts it into markers, using the place ID as the marker ID. Handle empty results and errors, and follow the service's attribution and data-use requirements. The sample-marker example can be run without a Places search request.

12 Programs and verification

Program	Expected behavior
00_number_format.dart	Three locale/pattern formatting examples.
01_http_request.dart	Loading, post title or failure with Retry.
02_area_calculator.dart	Validated rectangle and triangle areas.
00_fixed_map.dart	Fixed Beirut camera; pan and zoom.
01_user_location.dart	Location button, permission workflow and camera update.
02_markers.dart	Toggle two sample pins and read info windows.
main.dart	Combined map, location and sample pins.

Run `flutter analyze` and `flutter test` in each app; run `dart analyze` and `dart test` in the area package. Automated tests cover calculations, invalid dimensions, HTTP parsing and retry, and location permissions. They do not render a native map or verify an API key. Map configuration and real GPS behavior need a manual run on a configured target.

13 Corrections and common errors

- Do not delete a package's tests just to silence generated-test failures; replace them with checks of the public API.
- `TextEditingController` is a controller, not a widget. Own it in state and dispose of it.
- Invalid input is not automatically zero. Validate it before calculating.
- A package can contain several libraries; `part of` identifies one library, not the package.
- Package updates do not automatically propagate to all dependent apps. Constraints and lockfiles still apply.
- A compiler accepting an import does not prove that its plugin is registered on the chosen platform. Generate and configure the runner, then restart the app after plugin changes.
- The chapter's `Future<LatLng>` field must be initialized safely under null safety. Our location request is button-driven and avoids an uninitialized field.
- Do not assume a map key works for the Places web service. Enable and restrict each service separately.

14 Exercises

1. Find a package on `pub.dev` and explain its supported platforms, SDK requirement, license and setup steps.
2. Change the number-format example to display three mandatory decimal places. Explain how the pattern changes.
3. Add a circle-area function to the local package; validate the radius and add tests before updating the app.
4. Test blank text, negative values, `NaN` and very large dimensions in the calculator. Explain why display rounding is separate from calculation.

5. Add an HTTP test for invalid JSON and another for a response without a title.
6. Compare `pubspec.yaml` and `pubspec.lock`; identify a direct and a transitive dependency.
7. Build the fixed map first, then test location denial, approximate permission and disabled services.
8. Add a third sample marker with a unique ID and description; toggle the set twice.
9. Sketch the backend contract for a live nearby-search extension, including empty results and failure responses.

Official references

- [Flutter: Using packages](#); [Dart: Package dependencies](#).
- Package documentation: [http](#), [intl](#), [google_maps_flutter](#), [location](#).
- [Dart: Publishing packages](#).
- [Google Maps Flutter configuration](#); [Nearby Search \(New\)](#); [Maps API security guidance](#).

This document is maintained by Dr. Mohamad AOUE
Last updated: 2026-10-05